

BASTION LINE

Rapport Technique Officiel — V4

Bêta, qualité de vie, sauvegarde et harmonisation graphique

Date du rapport	27 septembre 2026
Dépôt	github.com/scuillerjack-oss/bastion-line-v0
Commit de départ (V3 vérifiée)	7191cea4b80808a031112344d995e5edc659161e
Commit final V4 code (HEAD distant vérifié)	b9ecb8efa76dc164d4acef4614182c0e4c20e327
Identifiant de build vérifié	v0.1.0+b9ecb8e (rebuild local GITHUB_SHA=HEAD, confirmé identique)
URL publique (bêta téléphone)	https://scuillerjack-oss.github.io/bastion-line-v0/
Statut	Candidate à la bêta physique V4 — NON VALIDÉE, décision humaine en attente

0. Audit de l'état réel avant toute modification

- Workspace : /home/user/bastion-line-v0, dépôt Git existant, branche main.
 - `git status` propre avant toute modification.
 - HEAD local = HEAD distant (`git ls-remote`) = 7191cea4b80808a031112344d995e5edc659161e, exactement le commit du rapport technique V3 (aucune divergence).
 - Suite de tests V3 relancée avant toute modification : 77 tests unitaires (67 hérités + confirmation) tous passants, job CI "build"/"deploy" du commit V3 déjà vérifiés success lors de la mission précédente.
 - Rapport technique V3 relu et confronté au code réel : verrou tactile en deux temps, direction archers, retrait de la Tour de contrôle, rééquilibrage et nouvelle icône sont bien présents et fonctionnels dans le code de départ.
-

1. Revente d'une tour

1.1 Conception

Taux de remboursement centralisé dans `TOWER_SELL_REFUND_RATE = 0.6` (`src/engine/constants.js`) -- référence V4 appliquée telle quelle, facilement ajustable en un seul endroit. La valeur investie (`getTowerInvestedValue`, `src/engine/towers.js`) additionne le coût de construction initial ET le coût de chaque palier d'amélioration RÉELLEMENT acheté -- jamais le seul coût de base -- garantissant qu'une tour améliorée se revend toujours plus cher qu'une tour brute, sans jamais rembourser 100% de l'investissement.

1.2 Interface : prix affiché AVANT confirmation, jamais de vente accidentelle

Le panneau d'amélioration d'une tour affiche désormais un bouton **Vendre (+N pièces)** -- le montant exact avant toute action. Le cliquer ouvre un sous-panneau de CONFIRMATION dédié (jamais un `window.confirm()` natif hors style du jeu), avec son propre délai anti-reflexe (300ms, cohérent avec le verrou tactile déjà validé en V3) : un clic trop rapide sur "Confirmer" est ignoré, seule une action réellement délibérée déclenche la vente. Après vente : la tour disparaît, l'emplacement est réellement libéré (une nouvelle tour peut y être construite immédiatement) et le montant crédité est EXACTEMENT celui qui avait été annoncé -- même fonction (`getTowerSellRefund`) utilisée pour l'affichage ET pour le crédit réel, jamais un calcul dupliqué qui pourrait diverger.

1.3 Vérification

8 tests unitaires (revente brute, revente améliorée plus chère que brute sans jamais atteindre 100%, chaque palier augmente le remboursement, montant crédité = montant annoncé, emplacement libéré et restructurable, tour réellement supprimée, vente d'un identifiant invalide sans crash, vente impossible après victoire/défaite) + 2 tests mobiles réels (revente brute avec vérification bout-en-bout du montant crédité et de la libération d'emplacement, comparaison brute/améliorée sur la vraie interface).

2. Lisibilité des améliorations

L'ancienne représentation empilait un petit anneau PAR palier possédé au-dessus de la tour -- au palier maximum (2 paliers d'amélioration), les deux anneaux très rapprochés se lisaient visuellement comme un symbole en "8", signalé en bêta comme "trop placeholder".

Remplacé par un indicateur UNIQUE : jamais plus d'une seule forme dessinée à la fois, donc structurellement incapable de reformer un "8". Un anneau CREUX tant que la tour n'a pas atteint son palier maximum ; un disque PLEIN, à la même position, une fois le palier maximum atteint -- une progression "creux → plein" immédiatement lisible sans ouvrir le panneau. Fonctionne identiquement pour les 3 familles, y compris la tour d'archers dont l'asset Leonardo protégé ne peut pas lui-même évoluer visuellement par palier (cahier V4, section 7).



Trois tours Canon aux paliers 0 (aucun indicateur), 1 (anneau creux) et 2/max (disque plein).

3. Bug de reprise de progression

3.1 Reproduction et diagnostic de la cause racine

Bug bêta : "après relance d'une partie, le jeu a ramené le joueur vers d'anciens niveaux ; après une nouvelle relance, le dernier niveau réellement atteint est réapparu." Ce motif exact -- régression PUIS auto-correction au relancement SUIVANT -- a d'abord été reproduit par un test unitaire dédié AVANT toute correction (`tests/save.test.js`) : deux instances chargent chacune leur PROPRE copie en mémoire de l'état de sauvegarde (`save`) ; l'instance B progresse et persiste un `lockedLevelIndex=3` ; l'instance A, plus ANCIENNE dans sa mémoire, écrit ENSUITE avec sa propre valeur périmée (1). Test exécuté sur le code de départ : **la progression persistée régressait bien à 1** -- bug confirmé et reproduit, pas supposé.

Cause racine : `markLevelUnlocked()` ne faisait confiance qu'à la copie EN MÉMOIRE de l'appelant pour décider d'écrire (`if (levelIndex > save.unlockedLevelIndex)`). Sur un appareil réel, plusieurs instances de l'app peuvent exister avec des états mémoire divergents : une PWA installée relancée depuis l'écran d'accueil Android peut réutiliser une tâche restée en arrière-plan au lieu de toujours réexécuter le script depuis zéro, ou une page peut être restaurée depuis le bfcache du navigateur sans jamais relire le disque. Si l'instance la plus ANCIENNE écrit APRÈS l'instance la plus RÉCENTE, sa copie périmée écrasait silencieusement la progression réellement la plus avancée.

3.2 Correctif appliqué à la cause

`markLevelUnlocked()` relit désormais systématiquement l'état RÉELLEMENT persisté juste avant d'écrire et ne retient que le MAXIMUM entre trois candidats (persisté, mémoire de l'appelant, nouvelle valeur) -- qu'importe quelle instance écrit en dernier, la progression ne peut donc plus jamais régresser. Renforcé côté LECTURE par une resynchronisation défensive au retour au premier plan (`src/main.js` : écouteurs `pageshow` avec `event.persisted`, et `visibilitychange`) qui relit l'état persisté et rafraîchit le menu si nécessaire.

3.3 Vérification

Le test de reproduction, réexécuté après le correctif, passe désormais (la progression reste à 3, jamais régressée). Un second test simule 5 cycles consécutifs RÉELS *progression* → *sauvegarde* → *rechargement du module* → *reprise*, vérifiant à chaque cycle que le niveau repris est exactement celui attendu. Côté mobile, un test dédié répète 4 cycles avec un VRAI `page.reload()` du navigateur (pas une simple réinitialisation en mémoire), confirmant que le bouton "Continuer"/"Jouer" et le niveau affiché après reprise sont systématiquement corrects.

4. Affichage du niveau

Le HUD affiche désormais "**NIVEAU N**" seul (jamais de total) au-dessus du compteur de vagues -- qui reste, lui, inchangé (VAGUE N/M), car il s'agit d'une information différente (la progression DANS le niveau courant, utile en combat) non visée par cette demande. Vérifié par un test mobile dédié confirmant le format exact et l'absence de tout total affiché.

5. Chantier graphique expérimental

5.1 Asset protégé -- respecté intégralement

La tour d'archers (asset Leonardo, public/assets/towers/tour_rapide_arbalette.png) n'a été ni modifiée, ni redessinée, ni remplacée. Elle a servi uniquement de RÉFÉRENCE de direction artistique, observée pour en extraire une palette et un vocabulaire visuel partagés : pierre grise en dégradé, bois/laiton chaud, ferrures bleu-gris, petite bannière bleue.

5.2 Éléments harmonisés

- **Base/forteresse** : corps de pierre désormais en DÉGRADÉ (au lieu d'un aplat), liseré doré sous les créneaux, porte encadrée de bois/doré, bannière recolorée en BLEU (au lieu d'orange générique) avec liseré doré.
- **Emplacements constructibles** : le socle elliptique plat est remplacé par une petite plateforme de pierre carrée/biseautée en écho, à petite échelle, au plinthe surélevé de la tour de référence, avec des accents dorés aux coins.
- **Canon** : base circulaire en dégradé de pierre avec anneau doré, tourelle recolorée en métal froid (cohérent avec les ferrures de la référence) au lieu d'un brun générique.
- **Longue portée** : spire en dégradé de pierre, bandeau doré, pointe métallique -- même vocabulaire que le canon et la base.



Base + emplacements constructibles harmonisés (détail réel).



Canon harmonisé (détail réel).



Vue d'ensemble : base, emplacements, canon, longue portée et tour d'archers (protégée, inchangée) dans la même scène.

5.3 Limites honnêtes (cahier V4, section 7 : "le signaler clairement plutôt que de masquer l'échec")

- **Écart de fidélité assumé** : l'asset de référence est un rendu 3D illustré (ombrage complexe, texture de pierre détaillée, perspective isométrique). Un rendu Canvas vectoriel 2D ne peut PAS atteindre ce niveau de détail ni ce rendu -- ce chantier rapproche la PALETTE et le VOCABULAIRE visuel (pierre/bois-laiton/métal froid/bannière bleue), pas le niveau de finition. C'est une cohérence de direction, jamais une reproduction de qualité.
- **Ennemis et décor NON retouchés cette passe** : décision assumée, pas un oubli. Les silhouettes d'ennemis sont gameplay-critiques (le cahier V4 lui-même interdit explicitement de dégrader leur lisibilité) ; les retoucher demandait une prudence et un temps de validation visuelle supplémentaires que cette mission, déjà large (revente, indicateur de palier, bug de reprise, affichage), n'a pas priorisés. Le terrain/décor reste également inchangé (cahier : "certains éléments SIMPLES du décor", non prioritaire face à la lisibilité gameplay).

6. Fichiers modifiés / ajoutés

Fichier	Nature
src/engine/constants.js	Modifié — TOWER_SELL_REFUND_RATE
src/engine/towers.js	Modifié — getTowerInvestedValue, getTowerSellRefund
src/engine/simulation.js	Modifié — sellTower()

src/engine/save.js	Modifié — markLevelUnlocked() concurrence-sûr (cause racine du bug de reprise)
src/main.js	Modifié — UI revente + confirmation, resync progression au premier plan, affichage niveau
src/ui/render.js	Modifié — indicateur de palier unique, harmonisation base/emplacements/canon/longue portée
src/ui/audio.js	Modifié — sfx.sell
src/style.css	Modifié — .sell-option, .cost.sell
index.html	Modifié — #level-label (affichage "Niveau N")
tests/save.test.js	Modifié — reproduction du bug de reprise + test de cycles consécutifs réels
tests/simulation.test.js	Modifié — 8 tests de revente
scripts/check-mobile.mjs	Modifié — 4 nouveaux tests (affichage niveau, revente brute, revente comparée, cycles de rechargement réel)

7. Tests et résultats complets

Suite	Résultat
Tests unitaires (node:test)	77/77 passants (67 hérités de V3 + 10 nouveaux)
Vérifications mobiles/PWA (Playwright)	26/26 passantes (22 héritées de V3 + 4 nouvelles)

Aucune assertion existante n'a été supprimée ni affaiblie. Non-régression vérifiée explicitement : vagues, combat, pause, victoire/défaite, navigation, sauvegarde/réglages, PWA/service worker, asset Leonardo (chargement + fallback), affichage mobile (5 largeurs, aucun débordement) -- tous les tests hérités de V0 à V3 passent inchangés.

8. Git, CI/déploiement — vérification réelle

Étape	Résultat vérifié
Commit final (code)	b9ecb8efa76dc164d4acef4614182c0e4c20e327
HEAD distant après push	Vérifié via git ls-remote — identique au commit local
Job CI "build"	success (77 tests unitaires + 26 vérifications mobiles exécutés en CI)
Job CI "deploy"	success — log lu directement, pages_build_version vérifié = commit ci-dessus
Build ID re-vérifié localement	v0.1.0+b9ecb8e (rebuild avec GITHUB_SHA=HEAD réel)
URL publique (lien de bêta téléphone)	https://scuillerjack-oss.github.io/bastion-line-v0/

9. Limites restant à valider sur téléphone physique

- **Revente** : logique et affichage vérifiés bout-en-bout en automatisé (montant affiché = montant crédité, confirmation anti-reflexe) -- reste à confirmer que le geste de confirmation se ressent bien comme volontaire sur un doigt réel, pas seulement en simulation Playwright.
- **Indicateur de palier** : lisibilité vérifiée par capture automatisée à taille réelle -- validation esthétique finale (le disque plein "se lit-il" instantanément comme "palier maximum" sans explication) réservée à la bêta physique.
- **Bug de reprise** : cause racine corrigée et verrouillée par des tests reproduisant précisément le motif observé (concurrence entre instances, cycles de rechargement réels) -- la confirmation définitive reste l'usage réel prolongé sur le téléphone physique du porteur, y compris après une vraie mise en arrière-plan Android prolongée.
- **Harmonisation graphique** : ce rapport ne déclare PAS le résultat esthétiquement réussi -- il documente ce qui a été tenté et ses limites assumées (section 5.3). Le jugement sur la cohérence réellement perçue appartient exclusivement au porteur.

10. Conclusion

Les cinq axes du cahier des charges V4 ont été traités comme une seule mission cohérente : une revente de tour complète et vérifiée bout-en-bout, un indicateur de palier unique corrigeant le défaut visuel signalé en bêta, un bug de reprise de progression reproduit puis corrigé à sa cause racine réelle (concurrence entre instances) plutôt que par un contournement, un affichage du niveau simplifié, et une première passe d'harmonisation graphique autour de la tour de référence protégée -- avec ses limites documentées honnêtement plutôt que masquées.

Conformément à l'instruction explicite du cahier, BASTION LINE V4 n'est PAS déclarée validée par ce rapport. Ce livrable est une candidate à la bêta physique V4. Le ressenti réel de la revente, la validité esthétique de l'harmonisation graphique, et la confirmation définitive du correctif de reprise sur un usage prolongé appartiennent exclusivement au porteur, après son propre test sur téléphone physique.